LxMLS - Lab Guide

June 18, 2026

Day 0

Sequence Models

0.1 Today's assignment

Today's class will be focused on advanced deep learning concepts, mainly Recurrent Neural Networks (RNNs). In the first day we saw how the chain-rule allowed us to compute gradients for arbitrary computation graphs. Today we will see that we can still do this for more complex models like Recurrent Neural Networks (RNNs). In these models we will input data in different points of the graph, which will correspond to different time instants. The key factor to consider is that, for a fixed number of time steps, this is still a computation graph and all what we saw on the first day applies with no need for extra math.

Although RNNs have been largely superseded by Transformer-based models for most NLP tasks, understanding them remains important: they introduce the core ideas of sequence modeling, backpropagation through time, and the vanishing gradient problem that motivated many subsequent architectural innovations, including Transformers.

If you managed to finish the previous day completely you should aim at finishing this as well. If you still have pending exercises from the first day e.g. the Pytorch part. It is recommended that you try to solve them first and then continue with this day.

0.2 Recurrent Neural Networks: Backpropagation Through Time

0.2.1 Feed Forward Networks Unfolded in Time

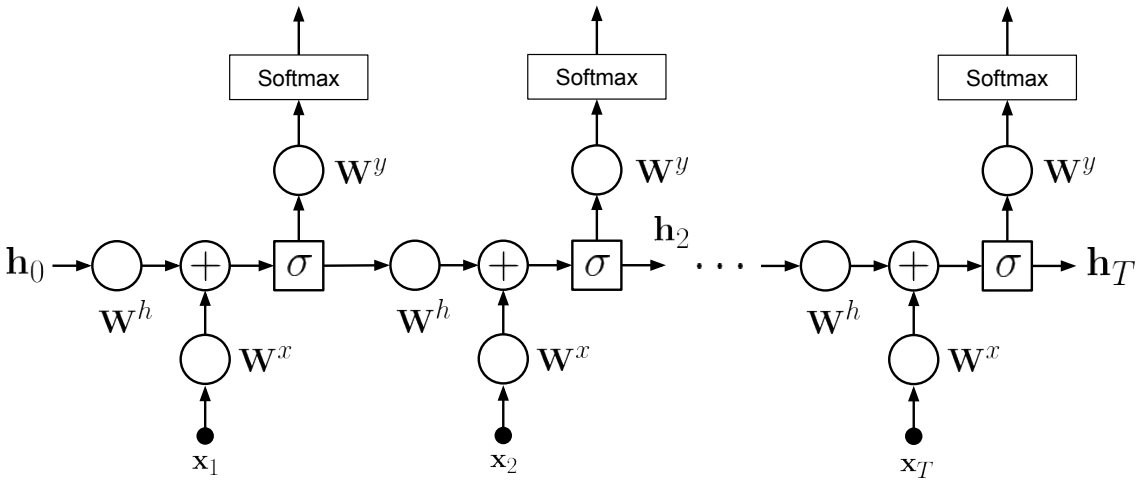

Figure 1: The simplest RNN can be seen as replicating a single hidden-layer FF network T times and passing the intermediate hidden variable $\mathbf{h}_t$ across different steps. Note that all nodes operate over vector inputs e.g,. $\mathbf{x}_t \in \mathbb{R}^I$. Circles indicate matrix multiplications.

We have seen already Feed Forward (FF) networks. These networks are ill suited to learn variable length patterns since they only accept inputs of a fixed size. In order to learn sequences using neural networks, we need therefore to define some architecture that is able to process variable length inputs. Recurrent Neural

Networks (RNNs) solve this problem by unfolding the computation graph in time. In other words, the network is replicated as many times as it is necessary to cover the sequence to be modeled. In order to model the sequence one or more connections across different time instants are created. This allows the network to have a memory in time and thus capture complex patterns in sequences. In the simplest model, depicted in Fig. 1, and detailed in Algorithm 1, a RNN is created by replicating a single hidden-layer FF network T times and passing the intermediate hidden variable across different steps. The strength of the connection is determined by the weight matrix $\mathbf{W}_h$

0.2.2 Backpropagating through Unfolded Networks

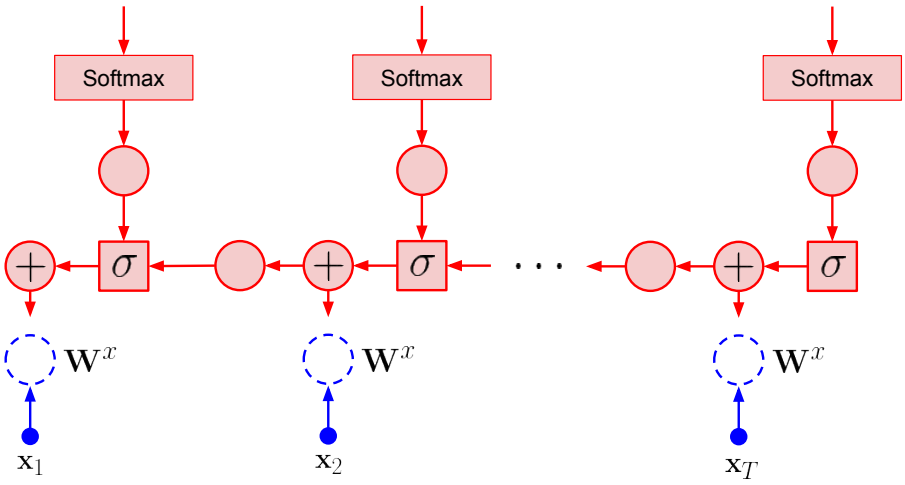

Figure 2: Forward-pass (blue) and backpropagated error (red) to the input layer of an RNN. Note that a copy of the error is sent to each output of each sum node $(+)$

It is important to note that there is no formal changes needed to apply backpropagation to RNNs. It concerns applying the chain rule just as it happened with FFs. It is however useful to consider the following properties of derivatives, which are not relevant when dealing with FFs

- When two variables are summed up in the forward-pass, the error is backpropagated to each of the summand sub-graphs
- When unfolding in T steps the same parameters will be copied T times. All updates for each copy are summed up to compute the total gradient.

Despite the lack of formal changes, the fact that we backpropagate an error over the length of the entire sequence often leads to numerical problems. The problem of *vanishing* and *exploding* gradients are a well known limitation. A number of solutions are used to mitigate this issue. One simple, yet inelegant, method is clipping the gradients to a fixed threshold. Another solution is to resort to more complex RNN models that are able to better handle long range dependencies and are less sensitive to this phenomenon, most notably the Long Short-Term Memory (LSTM) (Hochreiter and Schmidhuber, 1997) and the Gated Recurrent Unit (GRU) (Cho et al., 2014), both of which use gating mechanisms to control information flow across time steps. It is important to bear in mind, however, that all RNNs still use backpropagation as seen in the previous day, although it is often referred as *Backpropagation through time*.

The difficulty of training RNNs over long sequences was one of the key motivations behind the Transformer architecture (Vaswani et al., 2017), which replaces recurrence with self-attention and therefore has no path-length dependence on sequence length during gradient flow.

Exercise 0.1 *Convince yourself that a RNN is just an FF unfolded in time. Complete the backpropagation() method in NumpyRNN class in lxmls/deep_learning/numpy_models/rnn.py and compare it with lxmls/deep_learning/numpy/models/mlp.py.*

To work with RNNs we will use the Part-of-speech data-set.

```
# Load Part-of-Speech data
from lxmls.readers.pos_corpus import PostagCorpusData
```

Algorithm 1 Forward pass of a Recurrent Neural Network (RNN) with embeddings

1: **input:** Initial parameters for an RNN Input $\Theta = \{\mathbf{W}^e \in \mathbb{R}^{E\times I}, \mathbf{W}^x \in \mathbb{R}^{J\times E}, \mathbf{W}^h \in \mathbb{R}^{J\times J}, \mathbf{W}^y \in \mathbb{R}^{K\times J}\}$ embedding, input, recurrent and output linear transformations respectively.
2: **input:** Input data matrix $\mathbf{x} \in \mathbb{R}^{I\times M'}$, representing sentence of M' time steps. Initial recurrent variable $\mathbf{h}_0$.
3: **for** $m = 1$ **to** M' **do**
4: Apply embedding layer

$$z^e_{dm} = \sum_{i=1}^{I} W^e_{di} x_{im}$$

5: Apply linear transformation combining embedding and recurrent signals

$$z^h_{jm} = \sum_{d=1}^{E} W^x_{jd} z^e_{dm} + \sum_{j'=1}^{J} W^h_{jj'} h_{j'm-1}$$

6: Apply non-linear transformation e.g. sigmoid (hereby denoted $\sigma()$)

$$h_{jm} = \sigma(z^h_{jm}) = \frac{1}{1+\exp(-z^h_{jm})}$$

7: **end for**
8: Apply final linear transformation to each of the recurrent variables $\mathbf{h}_1 \cdots \mathbf{h}_{M'}$

$$z^y_{km} = \sum_{j=1}^{J} W^y_{kj} h_{jm}$$

9: Apply final non-linear transformation e.g. softmax

$$p(y_m = k|\mathbf{x}_{1:m+1}) = \hat{z}^y_{km} = \frac{\exp(z^y_{km})}{\sum_{k'=1}^{K}\exp(z^y_{k'm})}$$

```
data = PostagCorpusData()
```

Algorithm 2 Backpropagation for a Recurrent Neural Network (RNN) with embeddings

input: Data $\mathcal{D} = \{\mathcal{D}_1, \mathcal{D}_2, ..., \mathcal{D}_B\}$ split into B sequences (batch size 1) of size M'. RNN with parameters $\Theta = \{\mathbf{W}^e \in \mathbb{R}^{E \times I}, \mathbf{W}^x \in \mathbb{R}^{J \times E}, \mathbf{W}^h \in \mathbb{R}^{J \times J}, \mathbf{W}^y \in \mathbb{R}^{K \times J}\}$ embedding, input, recurrent and output linear transformations respectively. Number of rounds T, learning rate η

initialize parameters Θ randomly

for $t = 1$ **to** T **do**

  for $b = 1$ **to** B **do**

    for $m = 1$ **to** M' **do**

      Compute the forward pass for sentence b (M' time steps); keep not only $\tilde{z}^y_{km}$ but also, the RNN layer activations h_{jm} and the embedding layer activations z^e_{dm}.

    end for

    for $m = 1$ **to** M' **do**

      Initialize the error at last layer for a CE cost and backpropagate it through the output linear layer:

$$\mathbf{e}^m_y = (\mathbf{W}^y)^T \left(\mathbf{1}_{k(m)} - \tilde{\mathbf{z}}^y_m \right).$$

    end for

    Initialize recurrent layer error $\mathbf{e}_r$ to a vector of zeros of size J

    for $m = M'$ **to** 1 **do**

      Add the recurrent layer backpropagated error and backpropagate through the sigmoid non-linearity:

$$\mathbf{e}^m_h = \left(\mathbf{e}_r + \mathbf{e}^m_y \right) \odot \mathbf{h}_m \odot (1 - \mathbf{h}_m)$$

      where $\odot$ is the element-wise product and the 1 is replicated to match the size of $\mathbf{h}^n$.

      Backpropagate the error through the recurrent linear layer

$$\mathbf{e}_r = (\mathbf{W}^h)^T \mathbf{e}^m_h$$

      Backpropagate the error through the input linear layer

$$\mathbf{e}^m_e = (\mathbf{W}^x)^T \mathbf{e}^m_h$$

    end for

    Compute the gradients using the backpropagated errors and the inputs from the forward pass. For example for the recurrent layer

$$\nabla_{\mathbf{W}^h} \mathcal{F}(\mathcal{D}; \Theta) = -\frac{1}{M'} \sum_{m=1}^{M'} \mathbf{e}^m_h \cdot (\mathbf{h}_{m-1})^T,$$

    For the input layer

$$\nabla_{\mathbf{W}^x} \mathcal{F}(\mathcal{D}; \Theta) = -\frac{1}{M'} \sum_{m=1}^{M'} \mathbf{e}^m_h \cdot (\mathbf{z}^e_m)^T,$$

    Update the parameters, for example for the input layer

$$\mathbf{W}^x \leftarrow \mathbf{W}^x - \eta \nabla_{\mathbf{W}^x} \mathcal{F},$$

  end for

end for

Load and configure the NumpyRNN. Remember to use reload if you want to modify the code inside the rnns module

```
# Instantiate RNN
from lxmls.deep_learning.numpy_models.rnn import NumpyRNN
model = NumpyRNN(
    input_size=data.input_size,
    embedding_size=50,
    hidden_size=20,
    output_size=data.output_size,
    learning_rate=0.1
)
```

As in the case of the feed-forward networks you can use the following setup to test step by step the implementation of the gradients. First compute the cost variation for the variation of a single weight

```
from lxmls.deep_learning.rnn import get_rnn_parameter_handlers, get_rnn_loss_range

# Get functions to get and set values of a particular weight of the model
get_parameter, set_parameter = get_rnn_parameter_handlers(
    layer_index=-1,
    row=0,
    column=0
)

# Get batch of data
batch = data.batches('train', batch_size=1)[0]

# Get loss and weight value
current_loss = model.cross_entropy_loss(batch['input'], batch['output'])
current_weight = get_parameter(model.parameters)

# Get range of values of the weight and loss around current parameters values
weight_range, loss_range = get_rnn_loss_range(model, get_parameter, set_parameter, batch)
```

then compute the desired gradient from your implementation

```
# Get the gradient value for that weight
gradients = model.backpropagation(batch['input'], batch['output'])
current_gradient = get_parameter(gradients)
```

and finally call matlplotlib to plot the loss variation versus the gradient

```
%matplotlib inline
import matplotlib.pyplot as plt
# Plot empirical
plt.plot(weight_range, loss_range)
plt.plot(current_weight, current_loss, 'xr')
plt.ylabel('loss value')
plt.xlabel('weight value')
# Plot real
h = plt.plot(
    weight_range,
    current_gradient*(weight_range - current_weight) + current_loss,
    'r--'
)
```

After you have completed the gradients you can run the model in the POS task

```
import numpy as np
import time

# Hyper-parameters
num_epochs = 20

# Get batch iterators for train and test
train_batches = data.batches('train', batch_size=1)
dev_set = data.batches('dev', batch_size=1)
test_set = data.batches('test', batch_size=1)

# Epoch loop
start = time.time()
for epoch in range(num_epochs):

    # Batch loop
    for batch in train_batches:
        model.update(input=batch['input'], output=batch['output'])

    # Evaluation dev
    is_hit = []
    for batch in dev_set:
        is_hit.extend(model.predict(input=batch['input']) == batch['output'])
    accuracy = 100*np.mean(is_hit)
    print("Epoch %d: dev accuracy %2.2f %%" % (epoch+1, accuracy))

print("Training took %2.2f seconds per epoch" % ((time.time() - start)/num_epochs))

# Evaluation test
is_hit = []
for batch in test_set:
    is_hit.extend(model.predict(input=batch['input']) == batch['output'])
accuracy = 100*np.mean(is_hit)

# Inform user
print("Test accuracy %2.2f %%" % accuracy)
```

0.3 Implementing your own RNN in Pytorch

One of the big advantages of toolkits like Pytorch is that creating computation graphs that dynamically change size is very simple. In many earlier frameworks it was not possible to use a Python for loop with a variable length to define a computation graph. Again, we will only need to define the forward pass of the RNN and the gradients will be computed automatically for us.

Exercise 0.2 *As we did with the feed-forward network, we will now implement a Recurrent Neural Network (RNN) in Pytorch. For this complete the log_forward() method in lxmls/deep_learning/pytorch_models/rnn.py.*
Load the RNN model in numpy and Pytorch for comparison

```
# Numpy version
from lxmls.deep_learning.numpy_models.rnn import NumpyRNN
numpy_model = NumpyRNN(
    input_size=data.input_size,
    embedding_size=50,
    hidden_size=20,
    output_size=data.output_size,
    learning_rate=0.1
)

# Pytorch version
from lxmls.deep_learning.pytorch_models.rnn import PytorchRNN
model = PytorchRNN(
    input_size=data.input_size,
    embedding_size=embedding_size,
    hidden_size=hidden_size,
    output_size=data.output_size,
    learning_rate=learning_rate
)
```

To debug your code you can compare the numpy and Pytorch gradients using

```
# Get gradients for both models
batch = data.batches('train', batch_size=1)[0]
gradient_numpy = numpy_model.backpropagation(batch['input'], batch['output'])
gradient = model.backpropagation(batch['input'], batch['output'])
```

and then plotting them with matplotlib

```
%matplotlib inline
import matplotlib.pyplot as plt
# Gradient for  word embeddings in the example
plt.subplot(2,2,1)
plt.imshow(gradient_numpy[0][batch['input'], :], aspect='auto', interpolation='nearest')
plt.colorbar()
plt.subplot(2,2,2)
plt.imshow(gradient[0].numpy()[batch['input'], :], aspect='auto', interpolation='nearest'
    )
plt.colorbar()
# Gradient for  word embeddings in the example
plt.subplot(2,2,3)
plt.imshow(gradient_numpy[1], aspect='auto', interpolation='nearest')
plt.colorbar()
plt.subplot(2,2,4)
plt.imshow(gradient[1].numpy(), aspect='auto', interpolation='nearest')
plt.colorbar()
plt.show()
```

Once you are confident that your implementation is working correctly you can run it on the POS task using the Pytorch code from the Exercise 0.1.

Bibliography

Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.

Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8):1735–1780.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.